

Finding the Learning in Machine Learning

This piece is intended to be a window into not just the where or what, but most importantly the how — how you can explore machine learning and, in a broader sense, anything that you're curious about. The resources and domains mentioned here are merely directional and are not intended to be authoritative sources—mostly because I, too, am in the process of exploring this vast field, and it would be careless to view this article in isolation.

Most resources tend to start off with libraries such as `PyTorch` or `TensorFlow`. While these are incredibly useful tools, starting here can create the *illusion* of understanding machine learning. What has really happened is that you have become familiar with the API of a particular library, along with a rough understanding of the general process or template for building and training models. That is useful, but it does not necessarily equip you with the knowledge required to design your own machine learning architectures, models, or systems.

That brings me to what I believe is the more important question: do you want to use machine learning to build things, or do you want to build the machine learning systems themselves?

Depending on how you answer this, there are two broad approaches you could take.

The first is the application-oriented approach, which is primarily concerned with using existing machine learning techniques and tools to solve problems. The second is the fundamental approach, which is geared towards understanding the underlying principles of machine learning and its intricacies from the ground up.

This article will focus on the latter. The application-oriented approach is much more dependent on the application domain, where the nature of the problem, available data, constraints, evaluation criteria, and existing tools can introduce a large number of variables. Understanding the fundamentals, however, gives you a foundation from which those applications can be approached more deliberately.

I will be attaching links to resources along with any footnotes towards the end of this article, so make sure to glance over those when you're done reading.

The Three Musketeers

Before stepping into how you can get started with learning machine learning, it is important to distinguish between three closely related terms that are often used

interchangeably: Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL).

Let's start with Artificial Intelligence (AI). AI is the broadest of the three, concerned with building systems capable of performing tasks that we associate with aspects of intelligence, such as reasoning, perception, planning, decision-making, and language understanding. Machine learning is one of the approaches used to achieve this.

Machine Learning (ML) is a subfield of AI in which systems learn patterns, relationships, or decision functions from data or experience rather than having every rule explicitly programmed. There are many approaches to machine learning, ranging from traditional statistical and algorithmic methods to neural networks.

Deep Learning (DL) is, in turn, a subfield of machine learning that primarily uses neural networks with multiple layers to learn increasingly complex representations from data. The name "Deep" learning comes from the use of multiple layers of neurons which results in a sense of depth in the neural network. These layers allow neural networks to capture features at varying levels of detail and learn suitable representations between the input and outputs. **Convolutional Neural Networks (CNNs)**, **Recurrent Neural Networks (RNNs)**, and **Transformers** are examples of deep learning architectures, while **Generative Adversarial Networks (GANs)** are a framework built on such architectures, developed for different classes of problems.

A general theme is that a problem can be framed at an abstract level as an AI problem, and then can be narrowed down into a machine learning problem or deep learning problem by constraining it and approaching it in different ways.

Know Your Neighbors

Now that we have a clear idea of what machine learning is, we can get around to learning machine learning. When starting out, it may seem unreachable due to the sheer vastness of the concepts you need to understand. The good news is, you can start with a few of the fundamental concepts and progressively build towards a fuller understanding over time.

While we'll be taking a look at machine learning specifically, a first step to learning anything is to know the lay of the land and develop an understanding of the domain and related areas at a surface level. Machine learning in itself is a combination of a couple of different areas of study such as linear algebra, information theory, statistics and probability to name a few.

As a prerequisite, it would be helpful to have a good understanding of linear algebra and calculus; **matrices**, **vectors**, **eigenvectors and values**, **singular value decomposition**, **gradients**, and **differentiability** are some non-exhaustive

fundamental concepts that will let you learn machine learning without having to guess or gloss over concepts. For example, to understand the utility of certain loss functions or activation functions it is useful to know the idea of differentiability and how it translates to the mathematical feasibility of performing back-propagation.

Another example I can think of is the need for **bias** and an **activation function** in a neuron. The mathematical model of a neuron is often represented in terms of a set of **weights**, **biases** and an activation function. Inputs to the neuron are multiplied with the weights, biases are added to the result and the final result is processed by an activation function before it is passed onto the next neuron. The need for the **weights** is clear — it determines the impact or the influence of each input feature — but what about the **bias** and activation function? The purpose becomes clear only when you are aware of the mathematical consequences: **bias** shifts the activation threshold, and the **activation** introduces **non-linearity** — without it, even a deep stack of layers collapses to a linear mapping.

TLDR; mathematics is your best friend and it will haunt you for eternity ;)

Finding Nemo Learning

There are a few different paradigms for machine learning — supervised, semi-supervised and unsupervised learning (with others such as self-supervised and reinforcement learning expanding this family). Regardless of which paradigm is used, there are some common ideas that need to be cemented. We discuss these ideas below, along with pointers to resources you can use to grasp a better understanding.

Data! Data! Data!

🗣️ Garbage in Garbage Out

This single phrase underpins almost the entirety of machine learning. The quality of the data used to train a model dictates the quality of the model and its outputs. There is no way around this and as such, studying the data, cleaning it and identifying the most relevant attributes holds utmost importance.

While we're on the subject of data, I shall gloss over some important practices that revolve around data. Many of these concepts and practices are part of the broader **Machine Learning Development Life Cycle** — the **ML lifecycle** for short.

First, we need to source the data; if you're working on problems which have been studied for a considerably long time, then you may be in luck since platforms like Kaggle often have a wide range of datasets with different characteristics. On the other

hand, if you are looking at a fairly novel problem then you may have to collect data yourself or perhaps aggregate information from existing datasets to acquire what you need. All this being said, the perfect dataset does not exist and you will find yourself in a continuous pursuit for better data.

Once you have some data, the problem shifts to quality control, i.e., you need to clean/comb the data so that it is useful for training. This is often accompanied by **exploratory data analysis (EDA)** in order to identify outliers, redundancies, missing data and inconsistent scaling. **EDA** provides the metadata and analysis necessary to preprocess the data and turn into something much more valuable – information. The process of acting on the analysis from EDA and extracting useful information from the data is termed as **feature engineering**.

It may now be clear that more than anything, most of the time and effort in developing a machine learning system goes into acquiring, processing and extracting useful information from data.

The Machine

Now that we have some idea of the landscape surrounding machine learning, let's take a closer look at the machine itself. There are a lot of moving parts involved in training a model, but most of them can be understood by following a simple question: **how does the machine actually learn?**

The Model

At its core, a machine learning model is a function that takes some input and produces an output. In mathematical terms, we can loosely represent this as

$$\hat{y} = f_{\theta}(x)$$

where x is the input, $\hat{y}$ is the prediction, and θ represents the **parameters** of the model. In a neural network, these parameters primarily consist of **weights** and **biases**. The **architecture** of the model determines how these parameters are arranged and how information flows through them.

The machine doesn't inherently understand the information we give it, however. Images, text, audio and other forms of real-world information need to be represented in a mathematical form that the model can work with. This is where things such as **feature engineering**, **encoding**, **normalization** and, in modern deep learning, learned representations such as **embeddings** come into play.

The Forward Pass

Once the input has been represented appropriately, it can be passed through the model to produce a prediction. In a simple neuron, this might look something like

$$z = Wx + b$$

followed by an activation function,

$$a = \sigma(z)$$

A neural network essentially performs these transformations repeatedly across its layers. This process of taking an input through the model to produce an output is called the **forward pass**.

But producing a prediction isn't learning. We also need some way of determining how good that prediction was.

Knowing When It's Wrong

This is where the **loss function** comes in. A loss function measures the discrepancy between the model's prediction and the desired output:

$$L(y, \hat{y})$$

The choice of loss depends on the problem being solved. Predicting a continuous value, determining which of several classes an input belongs to, and generating a sequence of tokens are fundamentally different tasks and therefore require different ways of measuring error.

At this point, we have a prediction and we know how wrong it was. But there is still one important question: ***what should the machine change to become less wrong?***

The Learning

This is where the mathematics we encountered earlier starts to become useful.

The model contains parameters, and changing those parameters changes its predictions. We therefore want to know how sensitive the loss is to each parameter. This is expressed through gradients such as

$$\frac{\partial L}{\partial \theta}$$

which tell us the direction and magnitude in which a parameter influences the loss.

Backpropagation is the procedure used to efficiently calculate these gradients throughout a neural network. An **optimizer** then uses the gradients to update the model's parameters, with the learning rate controlling the size of those updates. A simplified gradient descent update looks like

$$\theta_{t+1} = \theta_t - \eta \nabla_{\theta} L$$

And this is, in essence, where the *learning* happens: the model repeatedly adjusts its parameters based on the errors it makes.

The Loop

Put everything together and the process becomes surprisingly simple:

This process is repeated over many batches and epochs until the model has, hopefully, learned a useful mapping between its inputs and outputs. The complexity of modern machine learning largely comes from the models, data, objectives and optimization procedures involved—not from this fundamental loop itself.

Of course, minimizing the loss on the data we trained on is not enough. A model that simply memorizes its training data has not necessarily learned anything useful. What we ultimately care about is **generalization**: how well the model performs on data it has not seen before.

This is where **train, validation and test sets**, evaluation metrics such as **accuracy**, **precision**, **recall** and **F1-score**, and concepts such as **overfitting** and **underfitting** enter the picture. Even the choice of metric is dependent on the problem domain; there is no universally meaningful definition of a "good" model.

Footnotes and The End?

As we come to the end of this ~~short~~ article I'd like to leave you with an idea and a few resources that should help you get started.

🗨️ Occam's Razor

Entities should not be multiplied beyond necessity.

Occam's Razor in the context of machine learning implies that the less complex an ML model, the more likely that a good empirical result is not just due to the peculiarities of the sample. This idea helps discern between two seemingly intelligent systems, as it is often the case that a **simpler model** has *learnt* well as opposed to a more complicated model.

Occam's Razor holds true for the process of learning machine learning or any other area of interest as well. A simple approach to learning can take you a long way ahead.

And finally, below is a collection of useful resources to get you going,

- <https://i.am.ai/roadmap> has some useful roadmaps for not just machine learning, but even AI and data science too.

- Google has some courses on machine learning at <https://developers.google.com/machine-learning> – I had started out with their foundational courses before venturing out into reading papers and learning advanced concepts
- If you're interested in understanding machine learning straight from the sources then reading research papers can be useful, however they can sometimes be quite heavy and dense – <https://paperswithcode.co/> is great place to explore and learn directly from research papers without being overwhelmed
- YouTube continues to be one of the best places to learn anything and is no different with machine learning
 - Channels like <https://www.youtube.com/c/CampusX-official> have playlists focused on different domains within AI
 - I acquired a vast majority of my knowledge and understanding in machine learning from this playlist: https://youtube.com/playlist?list=PLKnIA16_Rmvbr7zKYQuBfsVkjoLcJgxHH
 - <https://www.youtube.com/@deeplearningexplained> is yet another amazing channel that covers interesting deep learning concepts and research papers – his videos are great if you enjoy long videos that go into each and every detail with a mathematical perspective
 - Andrej Karpathy has a great playlist on neural networks (<https://www.youtube.com/playlist?list=PLAqhlrjkxbuWI23v9cThsA9GvCAUhRvKZ>) that covers back propagation and other foundational concepts with clear explanations
- You can find datasets for a wide range of problems on <https://www.kaggle.com/> and <https://huggingface.co/>
- While researching for this article I came across a few additional resources which maybe useful, although I haven't had the chance to explore them thoroughly
 - <https://www.openml.org/>
 - <https://projector.tensorflow.org/> has some cool visualizations of embeddings which can prove useful for understanding the visual implications of data when represented in various forms